AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 6.4: DEQN and OLG with Aggregate Uncertainty

Zhigang Feng

Spring 2026

Topic 6.4 Agenda

1. **Deep Equilibrium Nets (DEQN)**
2. **OLG with Aggregate Uncertainty**

Arc: T1 Aiyagari + computation → T2a Why ML / Euler → T2b Global DNN → T3 KS / DeepHAM → **T4 DEQN / OLG** *→ T5 MFG → Lab06*

Deep Equilibrium Nets (DEQN)

Deep Equilibrium Nets (Azinovic, Gaegauf & Scheidegger 2022, IER)

- DEQN is a **general** method: a **single** policy network, trained on equilibrium conditions, no value function. The original paper introduced it on **OLG / Bewley** economies.
- *In this lecture* we **apply** DEQN to the KS economy (the Lab06 setup). For KS the single network represents the policy:

$$\sigma_\theta(a, e, Z, K) \in (0,1) \;\Rightarrow\; a' = \underline{a} + \sigma_\theta \cdot (\text{cash on hand} - \underline{a}).$$

- The training objective is the **Euler equation residual**, not a Bellman residual.
- No value network, no fictitious play; just supervised learning on the Euler condition.
- Key innovation: a *cloud* of parallel economies provides a rich training distribution.

Why DEQN is the capstone of the deep-HA toolbox

DEQN combines the Euler-loss idea (Topic 6.2a) with the policy-network architecture (Topic 6.2b) — without ever computing a value function. Azinovic et al. (2022) demonstrate it on OLG/Bewley models; *this lecture* applies the same recipe to the full KS heterogeneous-agent benchmark.

DEQN: policy network architecture

- Single multilayer perceptron $\sigma_\theta : \mathbb{R}^4 \to (0, 1)$.
- Inputs: (a, e, Z, K) where $K = \int a \, d\mu$ is aggregate capital.
- Architecture applied in this lecture (Lab06):
  - 3 hidden layers, 64 units each.
  - Mish activation function.
  - Sigmoid output for the savings rate.
- Sigmoid output enforces the budget constraint at the network level: the implied a' always lies within $[\underline{a}, \text{cash on hand}]$.
- Contrast with DeepHAM: DeepHAM uses both V_{NN} and $\mathcal{C}_{\text{NN}}$; DEQN uses only the policy network.

DEQN: Euler equation as the loss function

The household optimality condition is the Euler equation:

$$u'(c_t) \;=\; \beta\,\mathbb{E}_t[(1+r_{t+1})\,u'(c_{t+1})]\,.$$

The **Euler residual**:

$$\varepsilon \;=\; 1 \;-\; \frac{\beta\,\mathbb{E}[(1+r')\,u'(c')]}{u'(c)}.$$

The DEQN loss aggregates squared residuals across a sample of agents:

$$\mathcal{L}(\theta) \;=\; \frac{1}{M}\sum_{m=1}^{M}\varepsilon_m^2.$$

Why Euler equations? They are *necessary* for optimality; sufficiency requires in addition a transversality / no-Ponzi condition (plus concavity). DEQN works because the bounded state space and simulated paths effectively impose transversality, so driving $\varepsilon \to 0$ trains the optimal policy without ever computing V.

DEQN: cloud simulation

Problem. To evaluate the Euler residual we need (K, Z), but K depends on the cross-sectional distribution μ, which itself evolves endogenously under the trained policy.

Solution. Simulate N *independent copies* of the entire economy in parallel:

- Epoch 0: all N economies start at the steady-state distribution μ_{ss} from a sequence-Jacobian (SSJ) calibration.
- As training progresses, each economy evolves under its own TFP path $\{Z_n\}$, so the set $\{K_n\}$ spreads out across a realistic range.
- Result: an economically consistent and *diverse* batch on which the Euler loss is meaningful.

Compared to DeepHAM (one economy, N agents, T_v-step rollout), DEQN runs N economies, each with its own μ_n, evaluating the Euler equation directly each step.

DEQN: distribution transport

At each epoch, $\mu_{n,t}$ is a histogram over (a_j, e_k). Steps to advance one period:

1. Compute savings $a'_{j,k}$ for every grid point under the current policy.
2. **Linear interpolation transport**: split mass $\mu(a_j, e_k)$ between the two adjacent grid points with weights $w_{\ell+1}, w_\ell$.
3. **Markov transition for employment**: distribute mass to (a_ℓ, e') weighted by the transition matrix $\Pi_{e \to e'}$.
4. Update $K_{n,t+1} = \sum_{j,k} a_j \cdot \mu_{n,t+1}(a_j, e_k)$.

The transport step runs inside `torch.no_grad()` so that gradients only propagate through the Euler residual loss, not through the simulation step itself.

DEQN: algorithm outline

Initialisation. Solve the steady state via SSJ to get K_{ss}, asset/employment grids, μ_{ss}, and the Markov transition Π.

For each training epoch $t \in \{1, \ldots, T\}$**:**

1. Compute $K_n = \int a \, d\mu_n$ and read Z_n for each cloud copy n.
2. Sample M random (a, e) per copy.
3. Forward pass: evaluate $\sigma_\theta \to (a', c)$.
4. Compute Euler residuals ε^2 via expectations over (Z', e').
5. Backward pass: Adam update on θ.
6. Transport: $\mu_n \to \mu'_n$ inside `no_grad()`; draw next Z_n.

Convergence. Loss decreases over ~ 2000 epochs; Euler errors below 10^{-3} indicate the policy solves the model to high accuracy.

Validation: Euler residuals over training

- Track $\log_{10}|EE|$ across iterations on a held-out validation sample of (a, e, Z, K).
- Standard targets (Judd 1998, applied to KS):
  - Acceptable: $\log_{10}|EE| < -3$
  - Good: $\log_{10}|EE| < -4$
  - In this lecture's KS run: **below** 10^{-3} **over** ~ 2000 **epochs**.
- **What healthy training looks like**:
  1. Loss curve monotone-decreasing on log scale (no oscillation).
  2. Aggregate K_n stays near steady-state range across cloud copies.
  3. Validation Euler error tracks training error (no overfitting — expected, since the loss *is* the optimality condition).

Validation: economic plausibility

- Beyond Euler errors, check that the *economics* is sensible:
  1. Savings policy $a'(a, e, Z, K)$ is monotone increasing in a for each fixed (e, Z, K).
  2. Employed agents save more than unemployed at the same a (precautionary motive).
  3. Aggregate K_n tracks the KS log-linear PLM slope-and-intercept reasonably.
  4. Wealth distribution under simulation matches Aiyagari-style steady-state shape (right-skewed, fat upper tail).

- **Diagnostic plots** for the lab:
  - Loss curve on a log scale.
  - Policy heatmap $a' = \sigma_\theta(a, e, Z, K)$ at fixed (Z, K).
  - Stationary wealth histogram.
  - Cloud trajectory of K_n over training (fan plot).

DEQN vs. DeepHAM: comparison

	DeepHAM	**DEQN**
Networks	Value + policy (2)	Policy only (1)
Optimality	Bellman / fictitious play	Euler residual
Distribution rep.	Generalised moments	Full histogram
Training data	N agents, 1 economy	N economies, cloud sim
Forward sim	Yes, T_v steps	Yes, 1 step / epoch
Distribution evolution	Implicit via moments	Linear-interp transport
Scalability	Moments fixed; NN width grows	Histogram sized to grid
Differentiability	Through truncated rollout	Euler loss only

- Both avoid the curse of dimensionality of the underlying KS problem.
- DeepHAM has richer distributional info via learned moments; DEQN has a simpler architecture and direct Euler-based training.

Recap: the modern HA toolbox

- Aiyagari (1994): a non-trivial nested fixed point, classically solved by VFI + bisection on r.
- Krusell–Smith (1998): adds aggregate uncertainty, classically tamed by bounded-rationality moments.
- DeepHAM: replaces moments with *learned* sufficient statistics; alternates supervised value learning and fictitious-play policy improvement.
- DEQN: drops the value function entirely; trains a single policy network on Euler residuals over a cloud of simulated economies.
- Both deep-learning methods generalise straightforwardly to:
  - Multiple assets (HANK)
  - OLG with aggregate shocks (next part)
  - Mean-Field Games in continuous time (Part X)

OLG with Aggregate Uncertainty

From infinite-horizon to overlapping generations

- In Aiyagari/KS, agents are infinitely lived. Many quantitative questions require finite life cycles:
  - Social security and pension reform
  - Retirement savings and 401(k)
  - Lifecycle wealth accumulation
  - Education financing and student debt
- **OLG model** (Diamond, Auerbach–Kotlikoff): agents live J periods (or cohorts), are born, age, and die.
- State vector now includes the *age* of each cohort, plus its asset/income distribution.
- Standard OLG with aggregate shocks remains computationally hard — the per-cohort distribution is itself an infinite-dimensional state.

OLG with aggregate uncertainty: state vector

With J cohorts indexed by age $j = 1, \dots, J$:

- Cohort j has its own asset distribution Γ_j over (a_j, y_j).
- Aggregate state: $S = (Z, \Gamma_1, \Gamma_2, \dots, \Gamma_J)$.
- Aggregate capital and labour:

$$K_t = \sum_{j=1}^{J} \int a \, d\Gamma_{j,t}, \qquad L_t = \sum_{j=1}^{J} \int \bar{\ell}_j \, y \, d\Gamma_{j,t},$$

  where $\bar{\ell}_j$ is the cohort-specific labour endowment (often hump-shaped).
- Lifecycle Bellman: $V_j(a_j, y_j, S) = \max\{u(c) + \beta \, \mathbb{E}[V_{j+1}(a', y', S')]\}$, with $V_{J+1} \equiv 0$ (or a terminal bequest function).

OLG state vector: budget constraints and lifecycle profile

Within each cohort, the household's intertemporal problem at age j has the budget constraint

$$c_j + a_{j+1} \;=\; (1 + r_t)\, a_j \;+\; w_t\, \bar{\ell}_j\, y_j, \qquad j = 1, \ldots, J,$$

with terminal condition $a_{J+1} = 0$ (no bequest) or $a_{J+1} = b$ (warm-glow bequest).

- **Age-profile labour endowment** $\bar{\ell}_j$ is hump-shaped: rising 25–50, declining 50–65, zero after 65 (retirement).
- **Cohort dimensionality.** With $J = 60$ cohorts and a 2-dim individual state (a, y), the full distribution $\{\Gamma_j\}_{j=1}^{J}$ lives in a $J \times 2 = 120$-dim space.
- **Backward induction (deterministic case).** Without aggregate shocks, solve $V_J \to V_{J-1} \to \cdots \to V_1$ cohort-by-cohort; aggregate shocks couple all cohorts simultaneously.

Why DL helps for OLG with aggregate shocks

- Without aggregate shocks: standard backward induction over cohorts works.
- **With** aggregate shocks: each cohort's continuation value depends on the future joint distribution of all cohorts $\Rightarrow$ exponential state-space blow-up.
- Deep neural networks let us:
  - Approximate $V_j(a, y, S)$ with a single MLP per cohort (or one MLP with cohort as an input).
  - Encode S via learned summary statistics (analogous to DeepHAM moments).
  - Train across cohorts in parallel on a GPU.
- Recent reference: Brumm, Kubler & Scheidegger (2023, working paper), "Economics-inspired neural networks with stabilizing homotopies".

Homotopy-stabilised training (Brumm–Kubler–Scheidegger 2023)

- OLG with aggregate uncertainty is hard to train: borrowing constraints, occasionally-binding bond limits, and discontinuities in the policy create unstable gradients.
- **Stabilising homotopy**: introduce constraints *gradually* during training.
  - Start with a relaxed (smooth) version of the model: no binding constraint, smooth penalties.
  - Slowly tighten the constraints toward their true value as training progresses.
  - By the time the constraints are at their true level, the network is already in a good basin.
- Empirically, homotopy is essential for getting OLG-with-aggregate-shocks training to converge.
- Implementation often combines JAX (for automatic differentiation through long simulation rollouts) with the Adam optimiser and a multistep homotopy schedule.

Homotopy schedule: results and convergence diagnostics

A typical 3-stage homotopy for OLG with aggregate uncertainty:

1. **Stage 1 — no aggregate shocks.** Fix $Z = \bar{Z}$; train on the deterministic OLG. Loss converges to $\sim 10^{-5}$ in a few thousand epochs.
2. **Stage 2 — small shocks.** Re-introduce aggregate shocks at $\sigma_Z^{(2)} = 0.3\,\sigma_Z^{\text{full}}$. Loss rises to $\sim 10^{-3}$, then converges.
3. **Stage 3 — full shocks.** Restore σ_Z^{full}; loss stabilises at $\sim 10^{-3}$.

Convergence diagnostics:

- Euler residuals below 10^{-3} on a held-out validation set.
- Cohort-by-cohort policy plots match the Stage-1 deterministic baseline at the modal aggregate state.
- Cross-stage warm-starting: each stage's final parameters initialise the next.

Why homotopy works: the policy never sees a discontinuous jump in the loss landscape.

Lab12_OLG.ipynb preview

- `Notebooks/Lab12_OLG.ipynb` is a JAX-based implementation of the Brumm–Kubler–Scheidegger OLG model with stabilising homotopies.
- Highlights:
  - 7 cohorts spanning a 72-period life cycle, life-cycle labour endowment.
  - Aggregate TFP shocks following an AR(1).
  - Capital adjustment costs.
  - State-dependent depreciation (TFP-tied).
  - Two-stage homotopy: capital-only model first, then add bond.
- Use this notebook as an **optional follow-up** after Lab06: students who finish the KS lab early can explore OLG.
- In production research, the same toolkit handles multi-asset HANK and asset-pricing models.

Next (T5a): the continuous-time view — Mean-Field Games, HJB, and the continuum-of-agents limit.